

ACSD: An Executable Evidence Profile for Evolving Pseudonymous Scholarly Releases

Anonymous authors

Abstract

Authors can already combine digital signatures, timestamps, and archival services to publish a pseudonymous manuscript. The central engineering challenge is shared interpretation: distributed tools and heterogeneous participants must agree on the exact objects being approved, the predecessor authority for a later revision, the point in time at which a multi-author approval set achieved completion, and the bounded conclusions justified by a subsequent disclosure. In uncoordinated workflows, primitive signing without explicit composition rules invites semantic drift, cross-context replay, and contextual overclaiming. We present ACSD, an executable evidence profile with a machine-checked decision kernel that makes these composition rules explicit. The reference implementation binds canonical release, author governance, evidence policy, and exact successor transitions to a joint approval target, while a separate acyclic approval set closes completed signatures before timestamping. A mechanized Lean development proves exact authorization, transition non-reuse, scope non-amplification, and refinement from restricted claim-free canonical JSON transcripts above trusted adapter facts to the abstract evidence model, with no `sorry`, `admit`, or project-specific axioms. Evaluation combines adversarial lifecycle corpora, cross-language differential verification, a scripted three-author coordination run, real RFC 3161 timestamps, 90 measured benchmark runs comparing ACSD with GnuPG and detached Ed25519, and a synthetic predicate-witness analysis that isolates one adversarial exposure for each of nine profile predicates in a finite corpus.

1 Introduction

Researchers often want to disclose a manuscript before conventional publication while keeping the author identity unavailable during review. A common informal proposal is to upload an anonymous file, sign it, and rely on repository history. That proposal leaves several distinct questions entangled.

One concrete trigger is the interval around an arXiv submission: a first-time or new-category submission may require endorsement, while moderation and review are separate from scientific peer review. During that interval an author may want a public, content-anonymous evidence package that preserves exact bytes and team assent alongside arXiv’s established submission process.

Why an explicit evidence profile is necessary. Consider three pseudonymous authors who release a manuscript, replace one team member in its next version, and later disclose only two identities. GPG can verify each detached signature, OpenTimestamps can witness a committed object’s time, and Zenodo can archive a release snapshot. However, these independent operations do not specify whether the prior team authorized the replacement, whether external time evidence covers every required approval or merely an unconfirmed proposal, or whether two slot disclosures justify a full-byline claim. An adversary can sign an apparent successor under a fresh key and obtain valid time evidence for it; primitive signature verification alone cannot determine whether that successor possesses legitimate lineage authority.

Our structured-signature baseline reproduces the central transition decision and establishes the mechanism-level lower bound. ACSD’s contribution begins at the composition boundary: a shared contract for object schemas, quorum aggregation, completion conditions, and permissible claim vocabularies. This contract lets independently implemented workflows reach the same decision instead of relying on verifier-specific interpretations.

ACSD (Anonymous Scholarly Claim and Disclosure) specifies this missing composition contract. The authoring path binds a release, its governance statement, its *Provenance Evidence Capsule* (PEC), and an exact successor transition to one approval target. Verification enforces predecessor authority rules and checks each derived claim against evidence of the correct type, slot, and scope. A distinct approval set closes over the completed signature quorum before timestamping, so that a timestamp can attest to the existence of the exact completed signature set rather than merely an unsigned target or incomplete collection. The public release preserves anonymity while author keys remain linkable across revisions.

Contributions.

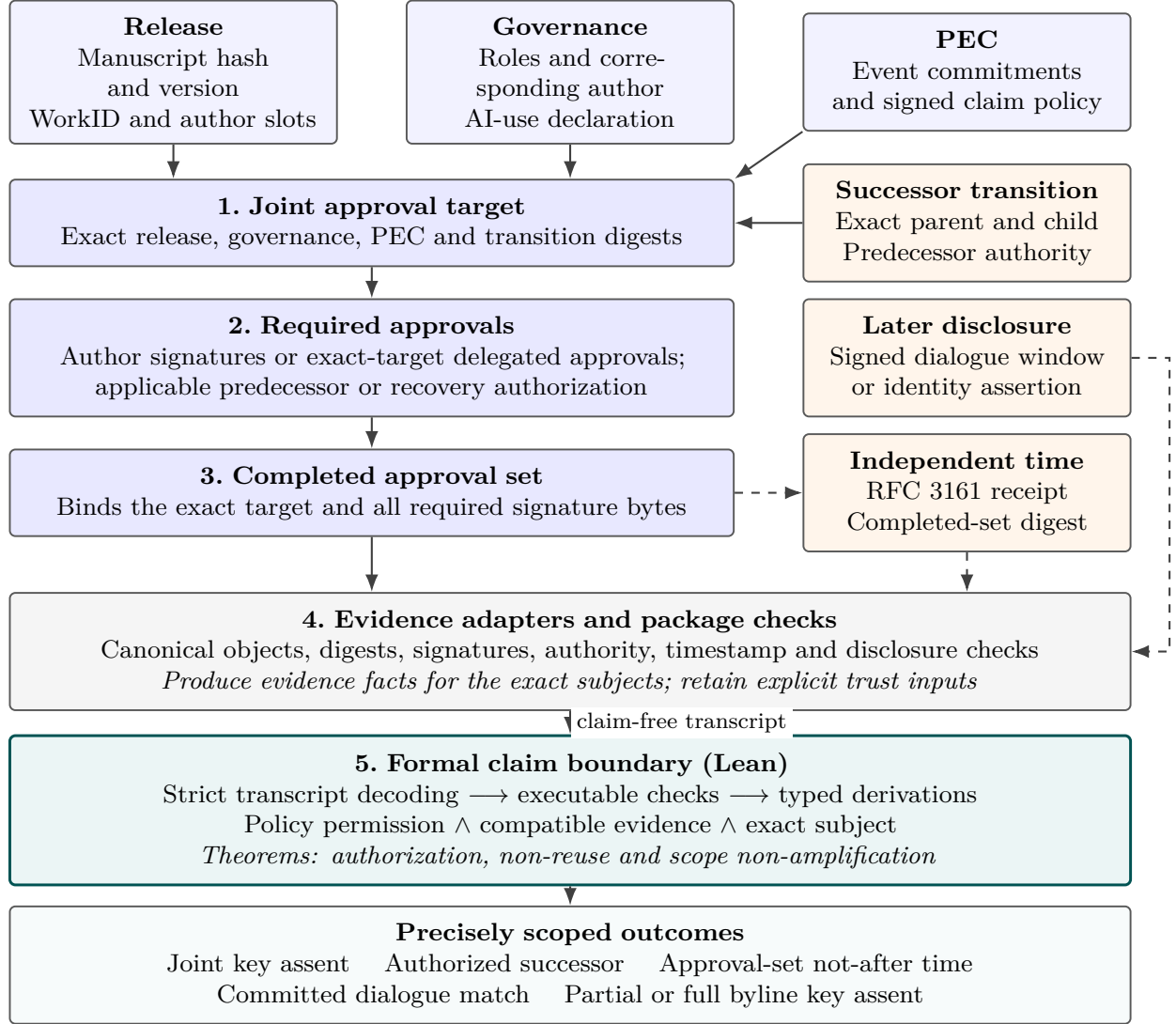
1. **A portable, versioned evidence profile.** We formulate an executable composition profile that specifies previously implicit decisions in evolving pseudonymous releases. Joint approval binds manuscript content, author governance, and evidence policies into an acyclic DAG. Successor authorization enforces predecessor authority and precommitted recovery, while scoped statements permit partial unblinding without mutating historical artifacts or compromising undisclosed keys.
2. **Machine-checked profile invariants and refinement.** Using Lean 4, we formalize the protocol rules and prove exact authorization, transition non-reuse, contextual scope non-amplification, and related soundness properties across 119 theorem and lemma declarations, with no `sorry`, `admit`, or project-specific axiom declarations. Refinement theorems connect the abstract evidence calculus to restricted, executable transcript checkers decoding Canonical JSON above trusted adapter facts across 69 positive and adverse decision cases.
3. **Evaluated implementation and predicate witnesses.** We deliver an open-source reference toolchain with multi-author workflows, coordinator-assisted aggregation, and real RFC 3161 timestamping. Evaluation combines 90 benchmarked operations against GnuPG and detached Ed25519, cross-implementation differential checks between Python and Node.js, and a synthetic predicate-witness analysis that isolates an adversarial exposure for each of nine tested claim-layer predicates.

2 Architecture and Evidence Flow

Figure 1 separates object construction, external evidence checking, and the formal claim boundary. The release digest fixes manuscript bytes and version metadata. Governance fixes author roles and declarations. The PEC commits evidence events and policy. For a successor, the transition also fixes the parent, child and proposed authority. All required keys approve the same target; finalization then fixes the exact signature set. These two stages distinguish assent to an object from evidence that the completed assent existed by a particular time.

Later timestamp receipts, dialogue openings and identity disclosures are separately checked against their original subjects. The adapter checks produce facts for the claim layer. The Lean model proves how those facts may compose, and the strict transcript checker reconstructs the

corresponding derivations. The adapter boundary and model assumptions are explicit; Section 8.4 states the principal machine-checked profile properties.



Solid arrows: construction or verification flow. Dashed arrows: optional time/disclosure paths.
The optional series layer resolves completed releases; it does not gate single-paper verification.

Figure 1: ACSD architecture from committed objects to justified claims. Successors use an exact predecessor-authorized transition; genesis has no transition. The completed approval set is the timestamp subject. Later disclosures retain their original PEC/event or release/slot scope. The Lean boundary derives claims above checked adapter facts. Diagram arrows show data flow.

3 Scope and Threat Model

Assets. A release contains manuscript bytes, an ordered list of anonymous author keys, roles and contribution declarations, an AI-use declaration, and optional parent-release information. Its approval set records the exact signatures that completed team assent. Later disclosure statements remain separate, subject-bound objects. Public keys are pseudonymous identifiers.

Adversary. We consider a party that can copy public content, generate fresh keys, reuse a WorkID and predecessor digests, or claim the next unused version. The party may reorder or substitute unsigned metadata, timestamp a target before adding signatures, replay a disclosure or transition under another scope, substitute an uncommitted recovery guardian, combine ordinary and recovery authorizations, or present a partial byline as complete. It may also reuse a key across works or submit signed but malformed JSON. Finally, a malicious or compromised author-slot key may sign a false identity string naming an uninvolved person. We assume neither that signatures reveal human identities nor that contribution declarations are true, recovery guardians remain honest or available, or local timestamps are trusted.

Scope and Boundaries. ACSD establishes cryptographic provenance and lineage authorization for digital releases. External attributes—such as natural-person legal identity, scientific correctness, originality, and peer-review status—inherently depend on external institutional evaluation and are orthogonal to cryptographic verification. For artifacts and branches presented to a verifier, the profile provides a tamper-evident record supporting post-publication review and dispute resolution; without an added transparency or gossip service, it neither proves that the presented history is complete nor discovers withheld branches.

4 Profile Acceptance Properties and Trust Boundary

The protocol is designed around four core acceptance properties that govern transitions and claims. Let $A(x, E)$ denote that the profile accepts object or conclusion x derived from evidence set E .

G1: successor authorization. If a child release is accepted as the successor of a given predecessor, E must contain an ordinary authorization by the applicable predecessor authority or an authorization by the exact recovery authority committed in advance. Knowing the WorkID, choosing the next version number, and signing under a fresh key are insufficient.

G2: exact-transition non-reuse. Authorization for a transition binds its parent, child, proposed authority and transition mode. Evidence accepted for one child cannot authorize a distinct child, even when both children reuse the same visible slot metadata.

G3: claim-type and scope integrity. Disclosed post-release evidence can justify only permitted claims of the matching type and exact subject. No combination of verified atoms can derive an outcome outside the declared policy vocabulary or widen the scope of an existing claim.

G4: completed-assent time evidence. Time evidence over an unsigned target establishes, at most, target existence. The stronger conclusion that the required approvals existed by a stated time requires evidence over the closed approval-set object and the applicable external-time checks.

These properties assume collision-resistant object identifiers, unforgeable signatures for uncommitted keys, correct results from the explicitly named adapters, and faithful evaluation of the model’s predicates. The Lean development proves these composition properties above verified adapter facts; standard cryptographic primitives (Ed25519, SHA-256) and external TSA services remain part of the trusted adapter boundary.

Table 1: Threat-to-decision boundary. “Rejected” refers to the presented package under the stated assumptions, not to global discovery of all events.

Adverse condition	Profile decision	Boundary or assumption
Fresh-key $n + 1$ child	Rejected as the same lineage without exact predecessor or precommitted recovery authorization	Ordinary signatures can implement the edge; ACSD standardizes its object, quorum, and derived outcome
Approval, transition, or disclosure replay	Rejected when type, purpose, audience, version, issuer context, or exact subject differs	Assumes the adapter reported the signed subject and signer correctly
Timestamp over an unsigned target	May establish target existence, not completed team assent	Completed-assent time requires a timestamp over the closed approval set
Withheld sibling or split view	No global winner is inferred; a conflict is reported when both branches are presented	Detection of withheld branches needs transparency, pinning, or gossip
Stolen slot key or false civil name	The signature may be cryptographically valid, but civil identity is not derived	Key custody, coercion, and identity authentication are outside the profile
Malformed parsing, primitive failure, or dishonest TSA	Outside the Lean result unless represented by a rejected adapter fact	Parsers, signature/hash libraries, PKIX policy, and TSA operation are trusted boundaries
Denial of service or evidence loss	No availability guarantee	Replication and operational recovery remain deployment concerns

5 Release Layer

5.1 Canonical release object

Each *PaperRelease* has a random stable *WorkID*, a visible slot (*WorkID*, *version*, *line*), a content path, content length and digest, ordered author records, declared roles, contributions, an AI-use declaration, optional exact predecessor, and a list of citation witnesses. The release identifier is

$$\text{ReleaseID} = \text{urn:sha256:H}(\text{canonical}(R)).$$

The content digest is over exact manuscript bytes. The release object uses a deterministic restricted JSON encoding: recursively sorted ASCII object keys, no insignificant whitespace, and safe integers only. The profile enforces this encoding strictly to guarantee cross-platform byte equivalence across independent verifiers.

5.2 Joint approval target and closed claim rules

The *AuthorshipGovernanceStatement* is the exact declaration of the work id, manuscript digest, ordered byline slots and roles, corresponding-author key, and AI-use field. The reference builder derives roles from the team configuration and release; Authority arises when every declared author key approves the governance digest through the common target described below, providing verifiable key assent to the specified roles, contributions, and disclosure policy.

The command-line builder accepts repeatable AI tool and purpose declarations and one-based author slots recording human review. It rejects a partial declaration that supplies a tool without a purpose or vice versa. These fields, like contribution terms, are approved inside the common target, ensuring that the complete author team cryptographically attests to the declared attribution.

Fine-grained links from selected dialogue events to particular version changes remain a separate disclosure workflow.

A *Provenance Evidence Capsule* (PEC) binds the release subject and governance digest to a digest-linked evidence-event sequence and a claim policy. The policy contains a fixed-vocabulary list of permitted outcomes, their required capabilities, and mandatory non-claims. A PEC also names one team key as an issuer for provenance, but that label alone grants no outcome: the complete team must still approve the common target.

The current command-line profile constructs an acyclic `acsd-approval-target/v2` object. It points to already complete objects and is itself signed only through detached approval envelopes. The following abbreviated schema excerpt shows its fields; angle-bracketed values are presentation placeholders rather than an accepted encoding:

```
{
  "schema": "acsd-approval-target/v2",
  "work_id": "urn:uuid:<work-id>",
  "release_digest": "<sha256>",
  "governance_digest": "<sha256>",
  "pec_digest": "<sha256>",
  "required_key_ids": [<key-id>],
  "lineage_transition_digest": null
}
```

Every required key issues a tagged COSE Sign1 signature over the same canonical target bytes. Verification recomputes all four bindings and each key id from the supplied public key. The closed claim rules determine

$$\begin{aligned} \text{Granted}(P, o) \iff & \text{Accepted}(P) \\ & \wedge o \in P.\text{permittedOutcomes} \\ & \wedge \text{CapabilityEstablished}(P, o). \end{aligned}$$

Policy permission is therefore only an upper bound; it is not evidence. The outcome vocabulary is closed and versioned, so an unknown social claim cannot enter the rule by adding a string to JSON.

Without the joint target, suppose authors sign only the release while an attacker later changes the unsigned governance object or PEC policy. The attacker could relabel roles, add the external-time outcome, or try to map a local clock to that outcome while the old release signatures still verify. Under ACSD, changing the governance or policy changes its digest and causes a target-binding failure; recomputing the target instead causes the old COSE approvals to fail their payload check. Even an approved policy entry yields no outcome without its separately verified capability. A verifier can thus state that the pseudonymous keys jointly approved this exact tuple; deployments that also require civil identity add separate enrollment evidence.

The approval target is deliberately unsigned data until its detached signatures arrive. Timestamping it at that stage would establish only *target existence*: an attacker who later obtains a key could append a valid approval after the recorded time. Finalization therefore constructs an `acsd-approval-set/v1` object containing the target digest and the SHA-256 digest of every exact author-approval COSE byte string, plus every separate predecessor-authorization signature used for an authority change. Only this closed approval-set object is the timestamp subject. Replacing, adding, or removing a signature changes the approval-set digest. The resulting evidence closure rule composes existing signature and timestamp primitives around one unambiguous completion subject.

5.3 Authorized succession rather than version-number possession

An exact predecessor digest proves structural continuity, not authority. A fresh-key attacker can copy a public WorkID, name the real predecessor, choose the consecutive version number, and

validly sign the resulting child with the attacker’s keys. No hash or slot-conflict check answers whether those keys were entitled to create the parent-to-child edge. The current profile therefore binds every release to a *lineage authority*: a sorted author-key set and a threshold for authorizing the next edge. The default threshold is unanimity. It is stored in the predecessor, so a child cannot lower it unilaterally.

Let P be an accepted predecessor, C a proposed child, A_P the authority bound by P , and A_C the authority proposed by C . The verifier accepts a lineage edge only if

$$\begin{aligned} \text{AuthorizedSuccessor}(P, C) = & \text{ExactEdge}(P, C) \\ & \wedge \text{Authorization}(P, C). \end{aligned}$$

For unchanged authority, authorization is

$$\text{Authorization}(P, C) = \text{Quorum}_P(\text{ApproveTarget}(C)),$$

where Quorum_P applies the author set and threshold fixed by P . For changed authority, let $T_{P,C} = \text{Transition}(P, C)$; then

$$\begin{aligned} \text{Authorization}(P, C) = & (\text{Quorum}(A_P, T_{P,C}) \\ & \vee \text{RecoveryQuorum}(R_P, T_{P,C})) \\ & \wedge \text{AllApprove}(A_C, C). \end{aligned}$$

Here **ExactEdge** requires the same WorkID, exact parent release and PEC digests, and either a same-line increment by one or a new named line beginning at version one. **Transition** binds both authorities and the exact child release, governance, and PEC digests. The child approval target binds the transition digest. The construction is acyclic because the transition refers to complete child bodies while detached approvals refer to their completed digests.

For an unchanged authority object, the ordinary child approvals already constitute predecessor authorization and no duplicate signing step is needed. A key-set or threshold change uses double control: the predecessor threshold signs the exact transition and every new author signs the child target. Here R_P is optional: the predecessor may commit a sorted, nonempty recovery-key set disjoint from A_P , together with its threshold. A recovery quorum can replace—not join—the online predecessor quorum only for an authority-changing child, and every child author still approves the exact child target. This resembles the old-and-new authorization pattern used for TUF root-key updates, but ACSD applies it to a single scholarly lineage edge rather than software repository metadata [15]. A series key may advertise a verified child as a head; it cannot manufacture predecessor-author assent by itself.

Consider the concrete $n+1$ attack. Mallory constructs a child with P ’s WorkID, P ’s exact release and PEC digests, the next version number, Mallory’s public key, and a valid Mallory signature over every child-controlled object. The object is internally signed but has neither an unchanged predecessor authority nor a transition signed by A_P . The implementation returns **VALID_OBJECT_BUT_UNAUTHORIZED_SUCCESSOR**, deterministically rejecting the unauthorized lineage transition.

This conclusion is relative to an accepted exact predecessor, not to the WorkID string alone. The CLI can pin a previously accepted parent with **-expected-parent-release-id**; otherwise it reports **UNPINNED_EXACT_PARENT**. Without an external parent anchor, an isolated package proves only that the authority named by its embedded parent authorized the edge; it cannot distinguish a separately fabricated genesis that copied a public WorkID.

Threshold choice exposes a real safety–liveness tradeoff. Unanimity prevents one coauthor from transferring the lineage alone but lets one lost or dissenting key freeze it. A lower threshold improves availability and is therefore allowed only when committed by the predecessor. A recovery set is a second, prospective tradeoff: it is public and linkable, must be committed before loss, and can authorize one exact authority-changing transition, but it cannot create a post-hoc identity recovery rule or revoke a valid past authorization. Without a precommitted recovery authority, lost threshold keys safely stop the cryptographic lineage. Two valid distinct children in the same exclusive slot, including one online-authorized child and one recovery-authorized child, are reported as equivocation with no automatic winner; choosing by observation time requires an external transparency or governance policy.

5.4 Optional citation and series extension

The release verifier does not depend on a series service. An optional extension records exact UTF-8 citation-token witnesses and lets a signed series bundle resolve stable WorkIDs to completed releases. This separates a semantic citation graph, which may contain cycles, from the digest and predecessor graph, which remains acyclic. The extension can detect an unsupported declared edge, but it does not decide whether a citation is intellectually appropriate or whether reused text is plagiarism. Because this facility is independent of the core publication, approval, revision, timing, and disclosure workflow, its detailed schema and graph certificate remain artifact documentation rather than a claimed contribution of this article.

6 Scoped Post-release Statements

Research-process evidence. Version lineage and committed dialogue records can preserve complementary parts of a research history: a release fixes the manuscript at one stage, while a later opening exposes selected records already committed by that stage’s PEC. For example, authors may reveal the dialogue behind an early conditional result while keeping unrelated conversations private, then use a later version for a stronger result and its new supporting material. Exact event and release bindings prevent moving an opening to a different subject. They establish neither that the opened window is representative of the complete research process nor that contrary committed material was also opened. An author can therefore construct a selective but cryptographically valid narrative by revealing only favorable windows. Whether the disclosed reasoning scientifically supports the stated result, and whether omitted context changes that assessment, remain reviewable content questions. The current prototype supplies the commitment and disclosure checks; automatic capture of research history, AI-use declarations and contribution-to-event mappings remains incomplete.

Some evidence is necessarily published after the immutable release: authors may open a committed dialogue window or reveal one name after review. Treating these as arbitrary metadata would reintroduce policy amplification. ACS D uses a small common rule for a statement E of type k , exact subject s , signer a , and requested claim c :

$$\begin{aligned} \text{Grant}(E, c) \Rightarrow & \text{SignatureValid}(E) \\ & \wedge \text{ScopeValid}(E, s) \wedge \text{Authorized}(k, s, a) \\ & \wedge c \in \text{Permitted}(k) \wedge \text{Compatible}(k, c). \end{aligned}$$

This shared skeleton is intentionally not a general policy language. Event, identity, and time evidence retain different schemas, authorization principals, and proof predicates.

For a dialogue event, the verifier derives the required signer set from the already approved PEC rather than trusting identifiers listed by the disclosure. It then checks the disclosure policy, exact PEC/event binding, contiguous Merkle opening, public-key identifiers, and every required COSE signature as one acceptance decision. A valid opening detached from its policy or a valid signature over a different opening grants nothing.

The slot-key identity assertion is an external per-slot sidecar binding the exact ReleaseID, WorkID, author slot, slot key, identity assertion, fixed purpose, and optional publication reference. Its result is named `SLOT_KEY_ASSENT_TO_IDENTITY_ASSERTION`: it establishes that the pseudonymous key of the designated slot signed the declared identity mapping. The deliberately narrow result says that the release slot’s pseudonymous key signed the declared mapping; it does not independently authenticate the named person or venue acceptance. In particular, possession of a stolen slot key is sufficient to create a protocol-valid assertion naming an uninvolved person. Consumers therefore must not render the result as authenticated civil identity without separate identity evidence. One slot can be revealed without disclosing others. A full-byline result requires a valid sidecar for every release slot; conflicting assertions for one slot are flagged as an unresolved dispute with no automated protocol winner.

The aggregate verifier labels any proper valid subset `PARTIAL_BYLINE_KEY_ASSENT`; it returns `FULL_BYLINE_KEY_ASSENT` only after quantifying over every slot in the accepted release. An optional fail-incomplete mode lets a publication workflow require full coverage while acknowledging that partial disclosure can be intentional. A common signed publication-reference string can crosswalk the released bytes to a later published DOI or repository record as an author-signed assertion. Extra or unregistered contribution fields are rejected by the strict parser.

Selective disclosure is cryptographically scoped but not socially isolating. Revealing some members of a team may permit inference about an undisclosed collaborator from public relationships, so a partial byline is not an anonymity guarantee for the remaining slots. Publication is also practically irreversible: observers cannot be made to forget an identity string, and the signature alone cannot reveal whether assent resulted from error, coercion, or key compromise. Authors should therefore treat identity and dialogue disclosure as deliberate publication, not as a reversible account setting.

Exact sidecar scope prevents cross-release replay but does not create cross-work unlinkability. Reusing the same public key across distinct WorkIDs exposes an equality edge. The built-in key-reuse audit reports this relation; independent keys per unrelated lineage reduce this linkage but do not hide account, network, or timing information.

6.1 Submission and unblinding lifecycle

A venue decision is deliberately not an ACSD state transition. Rejection can require no public action, leaving the previously released bytes unchanged. If review produces changed manuscript bytes, the submission becomes an exact predecessor-authorized successor. If the accepted text is byte-identical and only the byline is revealed, per-slot identity sidecars bind the existing release directly; manufacturing a redundant $n+1$ version would conflate identity disclosure with scientific revision. In either case, the authors may sign a publication reference, but only independently authenticated venue evidence could establish receipt or acceptance. Before publishing any sidecar, each participating author should inspect its exact release, slot and assertion; partial disclosure additionally requires a team-level privacy check because it may expose non-participating coauthors by inference.

7 Time Evidence

Time is intentionally external to author signing. An RFC 3161 request contains the SHA-256 imprint of the frozen approval set. A response supports the limited statement that the TSA signed a token for an imprint matching the locally recomputed complete target-and-signature set no later than its recorded `genTime`, only when the verifier supplies a TSA signer-certificate or fingerprint pin independently of the package. A certificate and its fingerprint copied into the same mutable package are not a trust anchor. The adapter checks the request/response nonce, SHA-256 imprint, `id-ct-TSTInfo` content type, single signer identifier, critical and exclusive timeStamping EKU, ESS certificate identifier, signed attributes, and CMS signature. It implements an exact-signer pin, not general PKIX path construction or revocation. Its local test TSA is a protocol test and can never produce `APPROVAL_SET_EXISTED_NOT_AFTER`.

There is no circularity: the timestamp receipt is a sidecar over the already closed approval set. A later revision can bind the receipt hash if a new signed object must refer to it. The timestamp provides cryptographic evidence that the completed approval set existed no later than the certified epoch.

Legacy releases timestamped only the approval target. They retain a distinct target-existence result and are never upgraded to an approval-set-time result. This typed distinction closes a temporal-confusion attack while preserving backward verification.

8 Implementation and Evaluation

The prototype uses Python for deterministic fixture generation and a zero-dependency Node.js verifier for strict CBOR/COSE and profile validation; the two paths are intentionally separate. The package is designed for offline replay: locked dependencies are included, and the verification driver regenerates fixtures, verifies a standalone release, builds a manifest, and verifies that manifest. The artifact also retains the optional series-profile tests, but they are not part of the focused workflow evaluated here.

A command-line tool (`acsd.py`) turns the profile into an executable workflow. Core commands are `keygen`, `init`, `author-approve`, `finalize`, and `verify`. Portable coauthor collection uses batch request/response directories and `coordinator-finalize`; the latter can exercise exact-target delegations addressed to supplied agent keys without receiving author private keys or requiring copied author identifiers. Lineage uses `authorize` and `revise`; a separate `compare-successors` command detects same-slot forks. The optional precommitted-recovery path uses `recover`. Unblinding uses `disclose-identity` and `verify-identity`; the aggregate command `verify-identity-set` distinguishes partial from full coverage. Finally, `audit-key-reuse` checks cross-work public-key equality. A one-command `release` path covers a locally held key set. Approval status is derived from verified COSE files, never trusted from mutable state metadata. COSE uses the registered EdDSA value `-8`, requires tag 18, and rejects non-minimal or ambiguous CBOR encodings. A zero-dependency Node path independently verifies Python-produced approvals. The canonical JSON layer is exercised by 64 fixed Python–Node vectors plus 1,000 seeded generated cases; all generated cases agree on canonical bytes and text-layer verdicts. Three known object-layer float typing differences do not change the text-layer rejection result and remain recorded in the canonical specification. The installable wheel and immutable paper/evidence directory are separate artifacts; the authoritative gate runs generated work in temporary directories and confirms source-tree byte identity afterward.

The core negative corpus includes canonical-JSON and path cases. Cross-layer regressions

Table 2: Typed evidence-to-claim confusion matrix. Checkmarks are the only derivable combinations; every off-diagonal cell is denied.

Evidence	Target time	Set time	Registered	Slot assent
Approval-target timestamp	✓	—	—	—
Approval-set timestamp	—	✓	—	—
SCITT inclusion	—	—	✓	—
Slot-identity assent	—	—	—	✓

replace a public key under an old key id; coherently replace governance and PEC; recompute an unsigned target and attempt to reuse old approvals; alter COSE tags, algorithm identifiers, map keys, and lengths; replace a timestamp and rewrite the transport manifest; mismatch the nonce, signer, EKU, TSTInfo content type, or signature algorithm; and attempt to promote the local test TSA. All are rejected at their intended boundary. One freeTSA receipt demonstrates that ACSID’s approval-closure validation interoperates with a deployed RFC 3161 service.

Additional regressions mutate approval-signature bytes after closure, attempt to relabel a target-only timestamp as approval-set time, detach a valid dialogue opening from its PEC or unanimous disclosure signatures, substitute a slot key, replay an identity mapping to another release, and promote a partial unblinding to a full byline. The deterministic demo carries real Ed25519 release-approval and disclosure-approval signatures rather than self-reported signer identifiers. Its verifier reconstructs the release approval target, checks every required release approval, then verifies the separately scoped event disclosure against the accepted PEC digest.

The local private-key profile is deliberately separate from the release wire format. **keygen-encrypt** writes an encrypted PKCS#8 Ed25519 PEM using the maintained cryptographic library and prompts twice for a passphrase on an interactive terminal. All signing commands share one loader: no passphrase is accepted through an argument, environment variable, JSON value, or release artifact, noninteractive encrypted-key use fails closed, and a one-command release or revision prompts once per private-key path. This protects exported private-key files at rest using standard encrypted PKCS#8 envelopes; stronger deployment profiles can connect the same signing interface to operating-system keystores or hardware-backed keys.

The executable semantic-confusion challenge applies four verified evidence kinds to four claim kinds. Only the diagonal combinations are derivable; all twelve cross-kind substitutions are denied. External social claims (e.g., natural-person civil identity, independent originality) are unconstructible in the closed derivation table. A separate timeline confirms that a target timestamp obtained before key compromise does not establish that approvals added after compromise existed at that earlier time.

The lineage corpus begins with the smallest discriminating attack: a child with the exact WorkID and parent release/PEC digests, $n+1$, a fresh attacker key, and complete self-approval. It is rejected at the missing predecessor-quorum boundary. Positive cases cover unchanged authority, key rotation, one-of-two and two-of-two thresholds, and a named branch; negative cases cover threshold downgrade and transition-signature replay. Two valid same-slot siblings yield equivocation with no winner.

An optional recovery corpus checks that only guardians committed by the parent can authorize one exact authority-changing child, that child authors still approve their target, and that ordinary and recovery methods cannot be mixed. An ordinary-authorized and a recovery-authorized child in the same slot remain a reported fork with no selected winner. Recovery is therefore an availability extension, not part of the minimum release and revision workflow.

Table 3: Executable evaluation summary. The artifact retains the complete case-level report, commands, and expected outcomes.

Evidence family	Count / result
Legacy and optional-extension fixtures	8 releases, 18 endorsements, 7 series objects; 30/30 checks
Canonicalization and manifests	64 fixed plus 1000 generated differential cases; 113 entries
Implementation regression suite	195 tests: 192 passed; 3 conditional skips locally; 0 failures or errors
Authorization and recovery	8 lineage cases; 6 recovery rejection classes plus an unranked fork
Post-release workflows	submission lifecycle, 1/3–3/3 unblinding, and cross-work isolation
Operational controls	scripted three-author run; manual exact-transition positive control
External-time interoperability	one real freeTSA response verified offline
Executable formal boundary	35 scoped, 15 lineage, and 19 appraisal Python–Lean decisions
Mechanized model	complete corpus builds without <code>sorry/admit</code>

8.1 Scripted distributed-author workflow

We additionally execute the production CLI as four isolated roles: three authors and one coordinator agent. The coordinator initializes an ordered three-author candidate and exports one self-contained request per slot. Two authors return direct approvals; the third signs a non-redelegable capability for the exact target and the distinct coordinator-agent key. One transactional `coordinator-finalize` invocation imports all responses, discovers the delegated slot from that agent key, exercises the capability, and finalizes the candidate. Offline verification then derives the narrower authorized-target approval outcome for the mixed set.

After key setup, the happy path comprises seven CLI invocations: initialization, batch export, three author responses, coordinator finalization, and verification. It requires no JSON edits, no private-key transfer, and no manual copying of a digest, WorkID, or key identifier. In an adverse run, the final returned signature is changed and its transport manifest regenerated. Batch import rejects the collection and leaves the live candidate with zero imported approvals. The runner contacts no external service and uses an explicit untimestamped test-only finalization policy.

This experiment validates executable role separation and transactional failure semantics across automated multi-author aggregation without requiring manual payload intervention.

To evaluate lineage controls against strong baselines, an experiment fixes the signature primitive to Ed25519 and evaluates three succession policies over identical parent and $n + 1$ candidates.

Recipe	Authors	Create (ms)	Verify (ms)	Extra bytes	Files
Detached Ed25519	1	307	282	177	3
GnuPG	1	315	110	348	3
ACSD	1	591	435	6,284	11
Detached Ed25519	3	252	226	531	7
GnuPG	3	653	246	1,044	7
ACSD	3	483	369	10,934	15

Table 4: Median local costs for a 1 MiB manuscript. Extra bytes exclude the manuscript. Each recipe’s cost is interpreted together with the application relations that it verifies.

Child-declared-key verification mistakenly accepts both legitimate rotation and fresh-key capture; naive predecessor-key pinning rejects both. A structured predecessor authorization over exact parent and child digests correctly permits rotation while rejecting unauthorized capture. ACSD unifies this authorization requirement with canonical scholarly objects, multi-party threshold governance, approval-set closure, and typed claim rules.

Three further executable workflows test post-release use. A two-author submission lifecycle leaves a rejected release byte-identical, creates an authorized successor only when manuscript bytes change, and binds an unchanged release to a later publication reference without deriving acceptance. A three-author unblinding run distinguishes 1/3 and 2/3 partial disclosure from 3/3 full coverage, and rejects duplicate slots, name mutation, and a smuggled contribution field. A cross-paper run rejects sidecar replay across releases while reporting that reuse of the same public key makes two WorkIDs publicly linkable.

8.2 Actual-client cost and security comparison

A separate local experiment compares the production ACSD CLI, an independent minimal detached-Ed25519 client, and GnuPG 2.4.9 (libgcrypt 1.12.2). The protocol fixes 64 KiB, 1 MiB and 8 MiB manuscripts and one or three authors. Each of the 18 configurations receives one warm-up and five measured repetitions, with shuffled recipe order: 90 measurements and 18 warm-ups. Timings include process startup and file I/O. Key generation and verifier key import are excluded setup operations. Both private and public keyrings are isolated test directories; GPG verification checks each required signer fingerprint.

Table 4 shows the cost of the declared recipes. For one author, verification sample standard deviations are 22, 28 and 40 ms for detached Ed25519, GPG and ACSD, respectively; for three authors they are 12, 22 and 25 ms. Creation standard deviations for the same recipes are 22, 54, 50, 9, 58 and 42 ms. The complete benchmark report retains every sample, all sizes, minima, maxima, standard deviations, commands and source hashes. The reported run is stored under `design/comparison-run-20260920-tcj`; its recorded production CLI hash matches the source evaluated for this manuscript.

The modest storage overhead reflects the application semantics: for the one-author 1 MiB case, ACSD takes about 1.9 times the detached Ed25519 creation time, 1.5 times its verification time, and stores about 6.1 KiB of structured metadata. With three authors, ACSD metadata is approximately 10.7 KiB, while median creation and verification times remain well below one second. This overhead corresponds to the additional capability to decide multi-author team assent, exact successor authority, precommitted recovery, completed-set time commitments, and scoped unblinding from a self-contained verifiable package.

Actual GPG checks accept the complete three-signer case, reject manuscript mutation and

a missing signature, and distinguish a self-declared attacker key from a pinned predecessor key. A GPG-signed transition confirms that statement-binding logic validates successfully and rejects altered child bindings. The lifecycle, partial-disclosure, and cross-release replay tests verify that ACSD enforces these application semantics in the tested release histories.

The real 4,649-byte RFC 3161 receipt was verified in two offline paths over five repetitions: receipt-only verification exhibited a median of 272 ms (SD 16), while checking the ACSD approval-set binding required 295 ms (SD 28). Sixteen associated receipt and approval-closure regression tests passed.

In-memory microbenchmarks record the observed growth of ACSD’s canonicalization and digest verification operations with object volume. Over five repetitions, processing 10, 100, 1,000, and 5,000 objects required median times of 2.23, 26.45, 227.54, and 718.81 ms, respectively, with peak traced memory at 5,000 objects of 7.04 MiB.

8.3 Synthetic Predicate-Witness Analysis

To test whether individual claim-layer checks make distinguishable decisions in our threat corpus, we conduct a parameterized predicate-witness experiment over a finite synthetic corpus of appraised evidence atoms (`research/claim_confusion/ablation.py`). The input atoms assume prior success of primitive cryptographic adapters (signature verification, digest computation, and timestamp receipt decoding). The evaluator is a compact, request-oriented executable form of the declared claim policy rather than a mutation of the production byte verifier. The evaluation measures whether disabling one application-level predicate admits a predeclared unauthorized request that the intact synthetic oracle rejects.

The parameterized synthetic oracle serves as the strong baseline. In the intact baseline, all four legitimate controls (L01: valid multi-author approval; L02: valid RFC 3161 timestamp; L03: valid transparency registration; L04: valid two-slot unblinding) evaluate to their expected derived claims, while all adversarial requests are strictly rejected. Table 5 reports the nine isolated ablations (A01–A09). Each run disables exactly one verification predicate while holding all other rules constant:

- **Type and context bindings (A01–A05):** Disabling envelope typing (A01), exact subject equality (A02), purpose/audience pinning (A03), policy versions (A04), or quorum identifiers (A05) enables a corresponding context substitution in the corpus. In particular, A02 allows an approval for Release *A* to count toward Release *B*, and A03 admits evidence carrying the wrong purpose and audience.
- **Quorum integrity (A06):** Omitting distinct-issuer verification allows an adversary possessing only one author private key to duplicate their signature, satisfying an m -of- n threshold alone.
- **Claim and closure boundaries (A07–A09):** Disabling typed output boundaries (A07) permits a timestamp-bearing evidence atom to support a stronger release-approval request that the intact typed rules reject. Disabling complete slot coverage (A08) enables a partial disclosure (e.g., one author revealing their key) to inflate into a full-byline endorsement claim. Disabling timestamp subject binding (A09) permits a valid timestamp receipt for an unrelated artifact to witness uncommitted approvals.

All four positive controls retain their expected derivations, the intact oracle rejects every adversarial request, and each isolated ablation produces the predeclared exposure with no other unexpected derivation in this finite corpus. The result supplies a distinguishing witness for the behavioral relevance of each tested predicate under the declared threat taxonomy. The analysis

ID	Disabled Oracle Predicate	Adversarial Exposure	Consequence Prevented by Intact Oracle
A01	Protected envelope type	M01 (Envelope confusion)	Rejects non-approval payloads (e.g., transparency receipts) misparsed as approvals.
A02	Exact subject binding	M02, M05 (Subject replay)	Prevents signatures on release <i>A</i> from authorizing release <i>B</i> or splitting quorums.
A03	Purpose and audience	M03 (Context substitution)	Rejects evidence whose declared purpose or audience does not match the requested claim.
A04	Exact policy version	M04 (Version substitution)	Prevents evidence appraised under one policy version from satisfying a request under another.
A05	Issuer quorum context	M06 (Quorum fragmentation)	Prohibits assembling quorums across disjoint or uncoordinated authority domains.
A06	Distinct-issuer counting	X01 (Signer inflation)	Precludes a single author key from satisfying multi-signature thresholds via duplicate inputs.
A07	Typed claim boundary	M07, M08 (Evidence overclaim)	Prevents timestamp or transparency-inclusion evidence from being relabeled as release approval.
A08	Complete slot coverage	M09 (Partial unblinding)	Bounds unblinding to disclosed slots, preventing a one-of-two reveal in the corpus from claiming a full byline.
A09	Exact timestamp subject	X02 (Witness substitution)	Binds RFC 3161 timestamps to exact approval-set digests rather than arbitrary target objects.

Table 5: Predicate-witness matrix over the synthetic claim-confusion corpus. The intact oracle admits all four positive controls (L01–L04) while rejecting eleven adversarial requests (M01–M09, X01–X02). Disabling each tested predicate admits at least one corresponding request that the intact oracle rejects.

operates at the policy layer after signature, digest, and timestamp adapters have appraised their inputs.

8.4 Machine-checked profile properties

Table 6 states representative profile invariants. The applicable ordinary or recovery quorum and already verified adapter facts are model inputs. The theorems establish these properties for modeled instances satisfying those assumptions.

The Lean development covers approval targets, approvals, digest-linked events, a closed outcome type, exact-subject scoped statements, lineage and recovery transitions, and claim-free appraisal transcripts. Its results fall into three categories: invariants projected from the profile’s acceptance rules; equivalence and soundness results between executable and declarative checks; and refinement results from strict transcripts to the abstract claim model. Together they establish, among other facts, that an approval for one target digest cannot be reused for a different target; every granted result has policy permission and compatible exact-subject evidence; all required approvals are present; and external time requires the approval-set subject, signer pin, nonce, RFC profile, and a non-local authority. The lineage theorems show that every accepted successor has the applicable authority evidence, a fresh authority cannot enter through the unchanged-authority path, and a transition bound to one child digest cannot authorize another child. The recovery theorems require

Adverse case	Machine-checked profile property
Reuse an approval	Approval binds its exact target; a different target cannot reuse it.
Capture the next version	Accepted succession has predecessor authorization or the specified precommitted recovery authorization.
Replay a transition	Authorization binds the exact child and cannot authorize another child.
Inflate a partial reveal	Full-byline assent requires every required slot.
Reuse evidence across scopes	Derived claims require compatible evidence for the exact subject.
Backdate completed assent	Target-only time evidence cannot derive approval-set time.

Table 6: Selected Lean-checked profile invariants. Several are direct safety consequences of the acceptance rules; their purpose is to make those rules auditable and to detect implementation drift across the maintained decision boundary.

a predecessor commitment, disjoint recovery authority, changed online authority, and an exact transition, while preserving the fact that a distinct ordinary-authorized edge is not retrospectively erased. The scoped layer proves that identity evidence cannot grant an event-opening claim, a missing slot prevents full-byline status, and a target-only timestamp cannot establish approval-set time. Evidence-set union cannot manufacture a capability: every derived claim still has compatible verified support in one component.

Unlike the earlier purely abstract boundary, Lean now strictly decodes the restricted canonical JSON for the maintained claim-free certificates and the generic `acsd-appraisal-transcript/v1` used by `acsd verify`. Its executable checker constructs exact-subject derivations, and refinement theorems map those derivations to the earlier digest-scoped abstraction for any chosen subject projection. Python and Lean agree on 35 scoped-certificate, 15 authorized-lineage, and 19 production-appraisal positive and adverse cases. The formalization targets the composition rules and claim-derivation logic, treating parsed cryptographic primitives and external adapter receipts as verified inputs. The source code contains no `sorry`, `admit`, or project-specific `axiom` declarations, compiling entirely within standard Lean 4 logic. The accompanying research artifact provides the complete theorem corpus by module, including automated build scripts and dependency audits verifying that all stated profile invariants hold without additional proof assumptions beyond the explicit model inputs.

9 Related Work

The profile builds on established foundations. PROV-DM supplies a general vocabulary for entities, activities, agents, derivations, and bundles [1]. Software Heritage identifiers provide intrinsic identifiers for software objects [2]; Trusty URIs provide hash-bearing immutable artifact identifiers [3]. COSE defines the signed binary envelope used for approvals [4]. SCITT specifies a broader architecture for signed statements and transparency services [5], while COSE Receipts standardizes receipt envelopes [6]; neither registration nor a valid receipt makes the registered statement true. RFC 3161 specifies the timestamp protocol [7]; RFC 9921 explicitly distinguishes COSE-then-timestamp from timestamp-then-COSE [8]. ACSD uses the former ordering as a semantic analogy: its separate approval-set object is timestamped after signature collection, without claiming wire-level interoperability with the RFC 9921 header profile. TUF uses predecessor and successor root thresholds to preserve trust through root-key updates [15]; in-toto binds supply-chain steps to authorized functionaries [16]. ACSD specializes related authorization ideas to an exact anonymous

scholarly WorkID succession rule.

C2PA Content Credentials bind signed provenance assertions to media assets and explicitly separate tamper validation from judgments about whether an assertion is good or true [11]. The CAWG Identity Assertion adds creator identity assertions and validation results for that ecosystem [12]. ClaimChain provides privacy-preserving authenticated claim repositories with non-equivocation goals [13]. These systems establish signed provenance, scoped identity, selective disclosure, and conflict resistance as existing design families. ACSD contributes a smaller scholarly-release profile that evaluates approval completion, revision authority, scoped disclosure, and executable adverse cases together.

SD-JWT defines selective disclosure for claims in a signed JSON payload [9], while BBS-based verifiable-credential suites additionally target unlinkable derived proofs [10]. For public linkage, ACSD instead publishes a direct link from an already public release-slot key to one identity assertion. This smaller mechanism avoids an issuer/holder credential layer, while the typed claim boundary leaves unlinkable credentials as a separable service level.

While standard detached signatures authenticate isolated payloads, they lack shared semantics for multi-party authority, lineage transitions, and closure. ACSD unifies these primitives into an integrated profile combining canonical scholarly releases, threshold author governance, closed-set timestamping, and formally checked claim scopes.

ACSD combines these established mechanisms into an executable scholarly profile for self-contained pseudonymous releases, multi-author approval, exact predecessor-authorized succession, completed-assent timing, and scoped post-release disclosure. Optional citation and series objects remain separable extensions. Appendix A summarizes the nearest-neighbor capability boundary.

10 Use of AI in the Research Process

Generative AI systems were used for protocol exploration, adversarial test design, implementation assistance, manuscript editing, and diagram preparation. No AI system is an author. The systems used were OpenAI Codex, Anthropic Claude, DeepSeek, and Google Gemini. The sole human author formulated the research questions, made the protocol and reporting decisions, reviewed the source and generated text, verified the formal builds and experimental reports, and assumes complete responsibility for this article.

11 Limitations and Future Work

ACSD deliberately proves and tests a bounded profile rather than an end-to-end identity or transparency service. Its principal limitations are as follows:

- **Trusted implementation boundary.** Lean checks the abstract rules and strict transcript refinements, not the filesystem, release builder, canonical JSON implementation, DER/CBOR and COSE parsers, RFC 3161 adapter, hash function, signature implementation, or external TSA. Those components remain explicit trusted inputs to the modeled decision.
- **Key custody and recovery.** Encrypted PKCS#8 protects an exported file at rest but does not prove human custody or resist host compromise. Precommitted recovery prevents post-hoc takeover within the model, yet does not prove guardian custody, revoke a valid historical transition, discover a withheld child, or choose a global head.
- **Time and global visibility.** Exact signer and message-imprint checks do not replace PKIX path construction, revocation, or long-term renewal when a deployment requires them.

Competing branches can be compared when presented, but global anti-equivocation requires additional transparency, pinning, or gossip assumptions. The single freeTSA receipt is an interoperability existence check, not a provider reliability study.

- **Identity and disclosure.** Slot-key assent is not authenticated civil identity, freedom from coercion, or proof that a selectively opened research record is complete. Partial disclosure may enable social inference about undisclosed collaborators, and published identity strings are practically irreversible. Introducing credential issuers or subject countersignatures would add a separate trust and revocation model that ACS D does not assume.
- **External validity and privacy.** The prototype has no independent human participants or production deployment. Its client comparison measures local GnuPG, detached Ed25519, and ACS D operations rather than OpenTimestamps or archival-service workflows. The predicate-witness analysis shares a threat taxonomy and policy oracle with its corpus; it establishes finite distinguishing examples rather than production-verifier minimality. Content-anonymous publication also leaves repository-account, network, timing, and public-key linkage outside the protocol.
- **Long-term maintenance.** Evidence surviving algorithm or certificate aging needs an explicit renewal policy, such as renewed timestamps or an Evidence Record Syntax scheme [17]. Relations across independently administered series and richer editor integration likewise remain future work.

12 Conclusion

ACS D turns the informal concept of an anonymous signed manuscript into an executable evidence profile with a machine-checked decision kernel. By explicitly binding release content, author governance, and successor transitions into an acyclic DAG, the profile rejects the modeled forms of cross-context replay and temporal overclaiming in presented workflows. A separate approval-set object enforces strict closure over multi-author signatures prior to external timestamping, while scoped sidecars enable selective identity and dialogue disclosure without modifying immutable release artifacts.

A mechanized Lean 4 development verifies selected authorization, non-reuse, scope, and refinement properties, with no `sorry`, `admit`, or project-specific axioms. Experimental evaluation includes 90 client benchmarks, one real RFC 3161 interoperability receipt, and cross-language differential tests of maintained interchange decisions. A separate finite synthetic predicate-witness corpus supplies distinguishing witnesses for nine claim-layer rules. Ordinary primitives supply the underlying cryptographic decisions; ACS D contributes a shared scholarly composition profile and executable boundaries on the claims derived from them.

References

- [1] W3C. *PROV-DM: The PROV Data Model*. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-dm/>
- [2] Software Heritage. *SWHID Specification: Core Identifiers*, version 1.2. https://www.swhid.org/specification/v1.2/5.Core_identifiers/

- [3] T. Kuhn and M. Dumontier. *Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data*. ESWC, 2014. https://doi.org/10.1007/978-3-319-07443-6_27
- [4] J. Schaad. *CBOR Object Signing and Encryption (COSE): Structures and Process*. RFC 9052, 2022. <https://www.rfc-editor.org/rfc/rfc9052.html>
- [5] H. Birkholz et al. *An Architecture for Trustworthy and Transparent Digital Supply Chains*. RFC 9943, 2026. <https://www.rfc-editor.org/rfc/rfc9943.html>
- [6] O. Steele et al. *CBOR Object Signing and Encryption (COSE) Receipts*. RFC 9942, 2026. <https://www.rfc-editor.org/rfc/rfc9942.html>
- [7] C. Adams et al. *Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)*. RFC 3161, 2001. <https://www.rfc-editor.org/rfc/rfc3161.html>
- [8] H. Birkholz, T. Fossati, and M. Riechert. *CBOR Object Signing and Encryption (COSE) Header Parameter for Timestamp Tokens as Defined in RFC 3161*. RFC 9921, 2026. <https://www.rfc-editor.org/rfc/rfc9921.html>
- [9] D. Fett, K. Yasuda, and B. Campbell. *Selective Disclosure for JSON Web Tokens*. RFC 9901, 2025. <https://www.rfc-editor.org/rfc/rfc9901.html>
- [10] W3C. *Data Integrity BBS Cryptosuites v1.0*. Candidate Recommendation Draft, 2026. <https://www.w3.org/TR/vc-di-bbs/>
- [11] Coalition for Content Provenance and Authenticity. *Content Credentials Technical Specification*, version 2.4, 2026. https://spec.c2pa.org/specifications/specifications/2.4/specs/C2PA_Specification.html
- [12] Creator Assertions Working Group. *Identity Assertion*, version 1.3, 2026. <https://cawg.io/identity/1.3/>
- [13] B. Kulynych, W. Lueks, M. Isaakidis, G. Danezis, and C. Troncoso. *ClaimChain: Improving the Security and Privacy of In-band Key Distribution for Messaging*. WPES, 2018. <https://doi.org/10.1145/3267323.3268947>
- [14] OpenTimestamps. *A timestamp proof standard for blockchain anchoring*. Project documentation, accessed 2026-09-11. <https://opentimestamps.org/>
- [15] The Update Framework. *The Update Framework Specification*, version 1.0.28. <https://theupdateframework.github.io/specification/v1.0.28/>
- [16] in-toto. *in-toto Specification*. <https://github.com/in-toto/docs/blob/master/in-toto-spec.md>
- [17] T. Gondrom, R. Brandner, and U. Pordesch. *Evidence Record Syntax (ERS)*. RFC 4998, 2007. <https://www.rfc-editor.org/rfc/rfc4998.html>

A Nearest-neighbor capability boundary

Table 7 maps each system to its native capability and locates ACSD’s composition boundary relative to that capability.

Table 7: Nearest-neighbor capability boundary.

System	Primary capability	ACSD boundary
Detached signature + exact transition	Exact-byte key control and hand-built successor authorization	Matches the core edge decision; application must define closure, thresholds, scopes, and claims
OpenTimestamps RFC 3161	Independent timestamp evidence	Establishes a committed imprint’s time under its trust model, not author roles or succession
Zenodo / SWHID	Versioned archival records and persistent intrinsic identifiers	Archives or identifies objects; ACSD binds pseudonymous team assent and authorized evolution
TUF / in-toto	Threshold key update and authorized supply-chain steps	Closest authorization analogue; ACSD specializes the decision to scholarly releases and disclosure claims
C2PA / CAWG / SD-JWT / BBS / Claim-Chain	Provenance, identity assertions, selective or privacy-preserving claims	Broader claim ecosystems; ACSD uses public slot sidecars and a closed scholarly claim boundary
ACSD	Release, governance, revision, witness and disclosure profile	Composes these decisions while retaining each backend’s trust assumptions